

Aula 9 - Programação Dinâmica

Programação Dinâmica (muito chamada de **DP**) consiste em uma técnica de programação muito utilizada em problemas de programação competitiva para resolver alguns problemas. Embora pareça difícil logo de cara, o princípio de DP pode ser resumido a **nunca calcular a mesma coisa duas vezes**.

Uma característica que define muito bem um problema de programação dinâmica é a **subestrutura ótima**. Isso quer dizer, basicamente que:

- “A solução ótima para um problema é construída a partir das soluções ótimas dos seus subproblemas”

Ou:

- “Uma solução para um subproblema é parte da solução do problema maior”

Isso é bem ambíguo a princípio, mas ao olhar pra alguns problemas, tudo fica mais palpável.

Problema: Fibonacci

Suponha que queremos consultar $F(n)$ para múltiplos valores de n , em que $F(n)$ é a recorrência de Fibonacci, definida por:

$$F(n) = F(n - 1) + F(n - 2) \quad F(0) = F(1) = 0$$

Fibonacci, da forma como implementamos quando vimos a primeira vez (recursivamente), tem complexidade $O(2^n)$. Se tentarmos calcular, por exemplo, $F(5)$ de forma puramente recursiva, a árvore de recursões fica mais ou menos como:

$$\begin{aligned} F(5) &= F(4) + F(3) \\ &= (F(3) + F(2)) + (F(2) + F(1)) \\ &= ((F(2) + F(1)) + (F(1) + F(0))) + ((F(1) + F(0)) + F(1)) \\ &= \left(((F(1) + F(0)) + F(1)) + (F(1) + F(0)) \right) + ((F(1) + F(0)) + F(1)) \end{aligned}$$

Notem que calculamos $F(2)$ mais de uma vez nessa chamada recursiva. Isso não é muito interessante quando pensarmos em casos maiores como $F(10)$ em que calcularíamos $F(4)$ várias vezes, por exemplo.

Como queremos múltiplas consultas, seria interessante melhorar isso, queremos partir da ideia de que, se eu já calculei $F(x)$ em algum momento, eu não preciso calcular de novo. Como faríamos isso?

Podemos instanciar o vetor `dp[]` de tamanho absurdamente grande, tipo `dp[10000]`, inicializado completamente preenchido como um valor padrão, tipo -1. Assim, podemos definir `dp[i]` como "o valor de $F(n)$ quando $n = i$ ". Assim, sempre que formos chamar a função $F(i)$, primeiro, verificamos se `dp[i] != -1`, se sim, usamos o valor contido em `dp[i]`, e poupamos uma porrada de cálculos redundantes. Se não, quer dizer que é a primeira vez que estamos calculando $F(i)$, e portanto precisamos calculá-lo e salvá-lo em `dp[]`.

O dessa Fibonacci ficaria mais ou menos assim:

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 10000;
vector<int> dp(MAXN, -1); // precisamos declarar o vetor dp globalmente

int fib(int n) {
    if (n == 1 || n == 0) return 1;

    // Se já calculamos, retornamos o valor salvo em O(1)
    if (dp[n] != -1) return dp[n];

    // Caso contrário, calculamos, salvamos e retornamos
    dp[n] = fib(n - 1) + fib(n - 2);
    return dp[n];
}
```

Dessa maneira, no pior caso, para chamar $F(n)$, se ainda não calculamos o valor de ninguém, teremos que calcular n valores, reduzindo nossa complexidade para $O(n)$.

Essa é uma das abordagens da função de Fibonacci em programação dinâmica, conhecida como **abordagem top-down** ou, mais comumente chamada de **memoização** (não, eu não escrevi errado, é assim mesmo). Essa é a abordagem mais intuitiva, dado que é só pensar na recursão normal e ir salvando valores para evitar de recalculá-los de forma redundante.

Existe, ainda, outra abordagem, chamada de **abordagem bottom-up** ou **tabulação**. Consiste no caminho contrário, partimos dos casos-base e vamos calculando até chegar no caso n .

```
#include <bits/stdc++.h>
using namespace std;

const int MAXN = 10000;
```

```
vector<int> dp(MAXN);

int fib(int n) {
    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}
```

Aqui, novamente, como cada valor só é calculado uma vez, temos $O(n)$.

Formalizando uma solução em DP

Via de regra, para resolver um problema com DP, precisamos definir:

1. O que `dp[i]` representa?
2. Como é calculado `dp[i]`?
3. Quem são os casos base?

Definidos esses tópicos, normalmente, temos tudo que precisamos para começar a codar.

No caso do problema anterior, por exemplo, definimos:

1. `dp[i]` representa o valor da função em i .
2. `dp[i]` é dado por `fib(n-1) + fib(n-2)` no caso da memoização;
é dado por `dp[i-1] + dp[i-2]` no caso da tabulação.
3. Os casos base são `dp[1] = dp[0] = 1`.

Problema: Prefix Sum

Esse problema também consiste de múltiplas consultas. Dado um vetor `v[]` com n inteiros, queremos fazer várias consultas. Cada consulta consiste de um valor m , queremos retornar $\sum_{i=0}^m v[i]$. Ou seja, a soma de todos os elementos de `v` do início até a posição m .

Se tivéssemos k consultas, no pior caso, cada consulta resultaria em $O(n)$ (somar todo mundo). Isso nos dá a complexidade de $O(kn)$. Como melhoramos isso?

Aqui, é bem tranquilo observar a subestrutura ótima, claramente, a soma de todos os números de 0 até i está contida na soma de todos os números de 0 até j se $j > i$. A chave aqui é usar DP para otimizar cada consulta.

Novamente, precisamos formalizar a solução em DP primeiro:

1. `dp[i]` nesse caso vai significar "a soma de todo mundo de 0 até o elemento i " (que é justamente o que queremos com cada consulta).
2. `dp[i]` nada mais é do que `dp[i-1] + v[i]` (ou seja, a soma de todo mundo até o elemento anterior, mais o elemento atual)
3. O caso base é quando a consulta é $m = 0$, que é só o primeiro elemento do vetor `v`.

Assim, ficaríamos com algo nessas linhas. Um vetor `v` e um vetor `dp` correspondente, tipo:

v:

4	2	0	5	6	1	2	3	-3	4	0	-2
---	---	---	---	---	---	---	---	----	---	---	----

dp:

4	6	6	11	17	18	20	23	20	24	24	22
---	---	---	----	----	----	----	----	----	----	----	----

Daí, calculamos o vetor `dp` inteiro apenas uma vez, e cada consulta vai retornar a soma de todo mundo de 0 até m em $O(1)$. Isso reduz a complexidade $O(kn) \rightarrow O(n)$.

A implementação, depois de formalizar tudo direitinho, é tranquila, então fica com vocês!

Problema: Caminhos em Grid

Dado um grid quadriculado de dimensões $n \times m$, com alguns obstáculos distribuídos nesse grid, de quantas maneiras conseguimos chegar à posição $(n - 1, m - 1)$ começando da posição $(0, 0)$? Assuma que, nesse problema, só podemos nos mover para direita e para baixo.

Vamos representar o grid como uma matriz de caracteres O ou X. O representa que não tem obstáculo ali, X representa que tem um obstáculo ali. Então, teríamos algo como:

$$grid = \begin{bmatrix} O & O & X & O & O \\ O & O & O & O & O \\ O & O & X & X & O \\ X & O & O & O & O \\ O & O & O & X & O \end{bmatrix}$$

Novamente vemos a subestrutura ótima: como só podemos vir de cima ou da esquerda, um caminho até uma determinada célula certamente depende de um caminho até a célula de cima, ou até a célula da direita.

Esse é um problema extremamente simples de uma DP de **duas dimensões**. Isto é, `dp` agora vai precisar de duas dimensões (ou seja, não temos mais um vetor `dp`, mas sim uma matriz `dp`).

Vamos formalizar os 3 aspectos da solução em DP:

1. `dp[i][j]` aqui vai significar o número de caminhos possíveis que temos para chegar na célula (i, j) . Vale ressaltar que se `grid[i][j] == 'X'` então `dp[i][j] = 0`.

2. O número de caminhos para chegar numa célula é exatamente o número de caminhos possíveis para chegar na célula imediatamente acima ou imediatamente à esquerda. Ou seja, calculamos `dp[i][j] = dp[i-1][j] + dp[i][j-1]` tomando bastante cuidado com a primeira linha e a primeira coluna da matriz (podemos acessar o índice -1 se não formos espertos)
3. O caso base aqui é a primeira célula do grid (0, 0), que só tem uma maneira de chegar lá (já estamos lá)

Daí, ao rodar o código no grid exemplo acima, teríamos o seguinte:

$$dp = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 2 & 2 & 2 & 2 \\ 1 & 3 & 0 & 0 & 2 \\ 0 & 3 & 3 & 3 & 5 \\ 0 & 3 & 6 & 0 & 5 \end{bmatrix}$$

Isso nos dá o número de caminhos de (0, 0) até (m-1, n-1) em $O(nm)$.